

Club Recommendation Graph ด้วย Neo4j

แนะนำชมรมจากทั้งเครือข่ายเพื่อนและความสนใจของนักเรียน

1. กำหนด Node และ Relationship

ใช้โครงสร้างดังนี้

- Node: Student, Club, Interest
- Relationship: FRIEND_OF (Student↔Student), MEMBER_OF (Student→Club), INTERESTED_IN (Student→Interest), FOCUSES_ON (Club→Interest)
- เป้าหมาย: แนะนำชมรมโดยดูทั้งจำนวนเพื่อนที่เป็นสมาชิก และว่าชมรมนั้นตรงกับความต้องการของนักเรียนหรือไม่

Pattern หลักสำหรับนับเพื่อน:

```
(me:Student)-[:FRIEND_OF]-(friend:Student)-[:MEMBER_OF]->(club:Club)
```

คำอธิบาย: เดินจากนักเรียนไปหาเพื่อน แล้วดูว่าเพื่อนแต่ละคนอยู่ชมรมใด ส่วน Interest จะใช้ในขั้นที่ 7 เพื่อเสริมเหตุผลการแนะนำ

2. สร้างข้อมูลตัวอย่าง (นักเรียน 6 คน ชมรม 4 ชมรม)

```
CREATE
(s1:Student {student_id:'S001', name:'Earth'}),
(s2:Student {student_id:'S002', name:'Yamal'}),
(s3:Student {student_id:'S003', name:'View'}),
(s4:Student {student_id:'S004', name:'Messi'}),
(s5:Student {student_id:'S005', name:'Kade'}),
(s6:Student {student_id:'S006', name:'Tar'}),

(c1:Club {club_id:'C001', name:'Music Club'}),
(c2:Club {club_id:'C002', name:'Painting Club'}),
(c3:Club {club_id:'C003', name:'Basketball Club'}),
(c4:Club {club_id:'C004', name:'Soccer Club'}),

(i1:Interest {name:'Strategy'}),
(i2:Interest {name:'Art'}),

(s1)-[:FRIEND_OF]->(s2),
(s1)-[:FRIEND_OF]->(s3),
(s1)-[:FRIEND_OF]->(s4),
(s1)-[:FRIEND_OF]->(s5),

(s2)-[:MEMBER_OF]->(c1),
(s3)-[:MEMBER_OF]->(c1),
(s4)-[:MEMBER_OF]->(c1),
(s5)-[:MEMBER_OF]->(c2),
(s6)-[:MEMBER_OF]->(c3),
(s1)-[:MEMBER_OF]->(c3),

(s1)-[:INTERESTED_IN]->(i1),
(c1)-[:FOCUSES_ON]->(i1),
(c2)-[:FOCUSES_ON]->(i2);
```

คำอธิบาย: รอบนี้ S001 (Nan) มีเพื่อน 4 คน (Two, View, Ployoung, Kade) แทนที่จะเป็น 3 คนแบบตัวอย่างเดิม และมีชมรมทั้งหมด 4 ชมรมแทน 3 ชมรม เพิ่ม Interest "Strategy" ที่ Nan สนใจ และ Chess Club ก็มุ่งเน้น Strategy เช่นกัน

Database information

Nodes (12)

* Club Interest Student

Relationships (13)

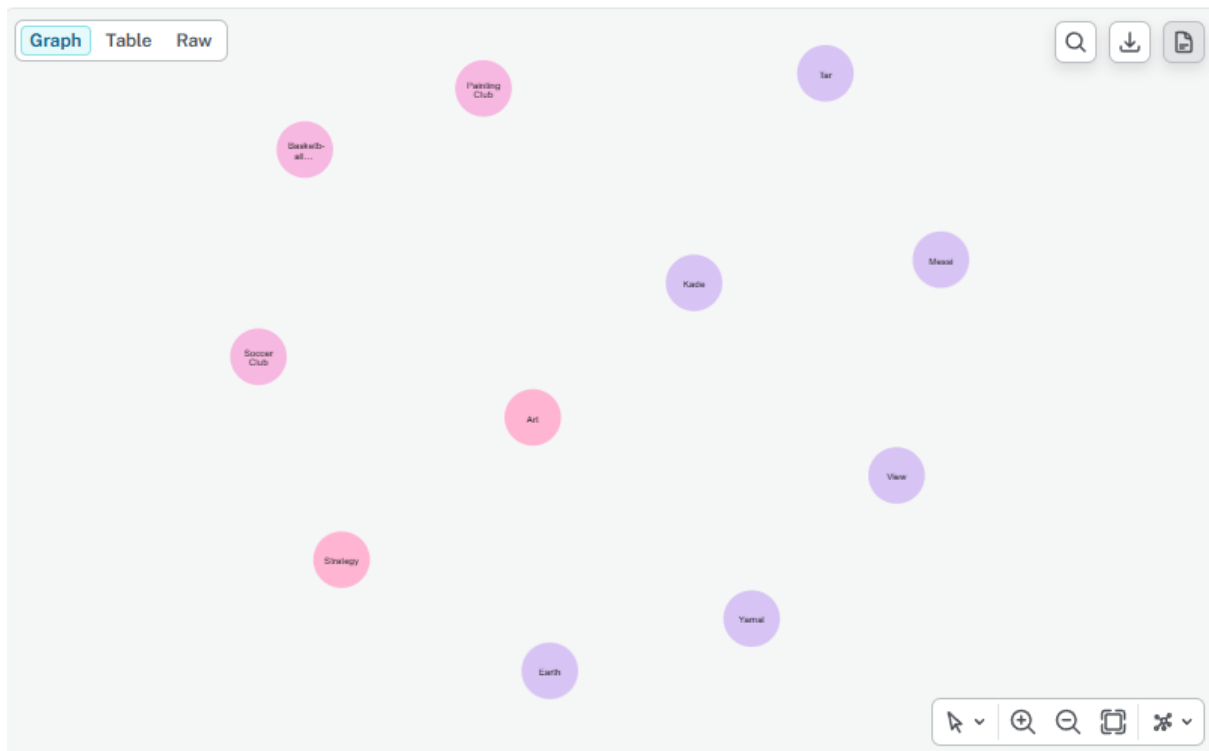
* FOCUSES_ON FRIEND_OF
INTERESTED_IN MEMBER_OF

Property keys

club_id name student_id

3. ตรวจสอบข้อมูลทั้งหมด

```
MATCH (n)
RETURN n;
```



คำอธิบาย: ผลลัพธ์ควรแสดง Node รวม 12 อัน (Student 6, Club 4, Interest 2) แสดงว่าสร้างข้อมูลสำเร็จ

4. ดูชมรมของเพื่อนนักเรียน S001

```
MATCH (me:Student {student_id:'S001'})
-[:FRIEND_OF]-
(friend:Student)
-[:MEMBER_OF]->
(club:Club)
RETURN friend.student_id AS Friend_ID,
friend.name AS Friend_Name,
club.club_id AS Club_ID,
club.name AS Club_Name;
```

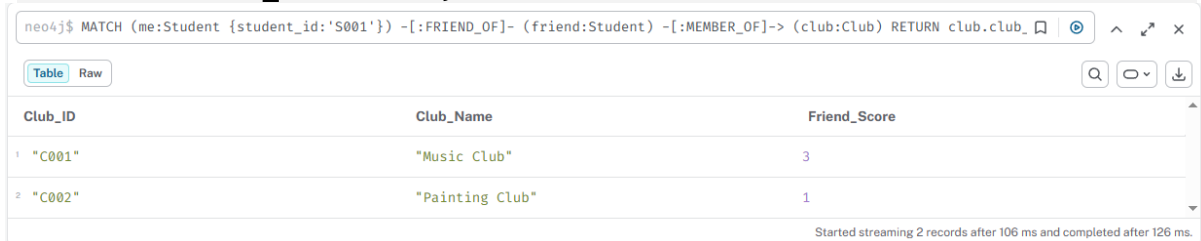
neo4j\$ MATCH (me:Student {student_id:'S001'}) -[:FRIEND_OF]- (friend:Student) -[:MEMBER_OF]-> (club:Club) RETURN friend.stu

Friend_ID	Friend_Name	Club_ID	Club_Name
1 "S002"	"Yamal"	"C001"	"Music Club"
2 "S003"	"View"	"C001"	"Music Club"
3 "S004"	"Messi"	"C001"	"Music Club"
4 "S005"	"Kade"	"C002"	"Painting Club"

คำอธิบาย: ครั้งนี้มีเพื่อน 3 คนอยู่ชมรมเดียวกัน (Chess Club) ต่างจากตัวอย่างเดิมที่กระจายกันมากกว่า จึงเห็นรูปแบบคะแนนที่ต่างออกไป

5. นับจำนวนเพื่อนในแต่ละชมรมเพื่อให้คะแนน

```
MATCH (me:Student {student_id:'S001'})
-[:FRIEND_OF]-
(friend:Student)
-[:MEMBER_OF]->
(club:Club)
RETURN club.club_id AS Club_ID,
club.name AS Club_Name,
count(DISTINCT friend) AS Friend_Score
ORDER BY Friend_Score DESC;
```



Club_ID	Club_Name	Friend_Score
1 "C001"	"Music Club"	3
2 "C002"	"Painting Club"	1

คำอธิบาย: `count(DISTINCT friend)` นับจำนวนเพื่อนที่ไม่ซ้ำกันในแต่ละชมรม Chess Club ได้คะแนนสูงสุด 3 เพราะมีเพื่อนถึง 3 คนเป็นสมาชิก

6. ตัดชมรมที่ S001 เป็นสมาชิกอยู่แล้ว

```
MATCH (me:Student {student_id:'S001'})
-[:FRIEND_OF]-
(friend:Student)
-[:MEMBER_OF]->
(club:Club)
WHERE NOT EXISTS {
    MATCH (me)-[:MEMBER_OF]->(club)
}
RETURN club.club_id AS Club_ID,
club.name AS Recommended_Club,
count(DISTINCT friend) AS Friend_Score
ORDER BY Friend_Score DESC;
```



Club_ID	Recommended_Club	Friend_Score
1 "C001"	"Music Club"	3
2 "C002"	"Painting Club"	1

คำอธิบาย: ต่างจากตัวอย่างเดิมตรงที่ Nan เป็นสมาชิก Basketball Club (C003) อยู่ก่อนแล้ว แต่ไม่มีเพื่อนคนไหนอยู่ชมรมนั้น เงื่อนไข `WHERE NOT EXISTS` จึงไม่ตัดอะไรออกจากลิสต์นี้เช่นกัน แต่ยังจำเป็นไว้ป้องกันกรณีทั่วไป

7. ใช้ Interest เสริมเหตุผลของอันดับ 1

```
MATCH (me:Student {student_id:'S001'})-[:INTERESTED_IN]->(i:Interest)
<-[:FOCUSES_ON]-(club:Club)
RETURN club.name AS Club_Name, i.name AS Shared_Interest;
```



Club_Name	Shared_Interest
"Music Club"	"Strategy"

คำอธิบาย: ขั้นนี้เพิ่มจากตัวอย่างเดิม เพื่อตรวจสอบว่าชมรมที่มี Friend_Score

สูงสุดตรงกับความสนใจของนักเรียนเองด้วยหรือไม่ ในที่นี้ Chess Club ตรงกับ Interest "Strategy" ของ Nan พอดี จึงเป็นหลักฐานเสริมอีกชั้นว่าเหมาะสมกว่าคำแนะนำจาก Friend_Score เพียงอย่างเดียว

8. สรุปชมรมแนะนำอันดับ 1

```
MATCH (me:Student {student_id:'S001'})
-[:FRIEND_OF]-
(friend:Student)
-[:MEMBER_OF]->
(club:Club)
WHERE NOT EXISTS {
    MATCH (me)-[:MEMBER_OF]->(club)
}
RETURN club.club_id AS Club_ID,
club.name AS Recommended_Club,
count(DISTINCT friend) AS Friend_Score
ORDER BY Friend_Score DESC
LIMIT 1;
```



Club_ID	Recommended_Club	Friend_Score
"C001"	"Music Club"	3

คำอธิบายผลลัพธ์:

ระบบแนะนำ Chess Club เป็นอันดับ 1 เพราะมีเพื่อนของ S001 เป็นสมาชิกถึง 3 คน (Two, View, Ploung) มากกว่า Painting Club ที่มีเพื่อนเพียง 1 คน (Kade) และยังตรงกับ Interest "Strategy" ที่ Nan สนใจอีกด้วย จึงเป็นคำแนะนำที่มีหลักฐานสนับสนุนสองทาง คือทั้งจากเครือข่ายเพื่อนและความสนใจส่วนตัว

Basketball Club (C003) ไม่ปรากฏในผลการแนะนำเพราะ Nan เป็นสมาชิกอยู่แล้ว ส่วน Choir Club (C004) ไม่ปรากฏเลยเพราะไม่มีทั้งเพื่อนหรือ Interest ที่เชื่อมโยงกับ Nan ในข้อมูลชุดนี้